

practical8

Q5_AWTForm_Events.java

```
// Import TextField: the text field whose contents are read/cleared by the handlers.
import java.awt.TextField;
// Import(ActionEvent): the object that carries information about a button click.
import java.awt.event.ActionEvent;
// Import ActionListener: the interface that must be implemented to receive button click events.
import java.awt.event.ActionListener;

/**
 * Practical 8 - Event handling for the Q5_AWTForm (Login Form).
 *
 * Contains ONLY event handling logic - no GUI code.
 * Attaches to the "Login" and "Reset" buttons of the login form.
 *
 * Behaviour:
 * - "Login": validates the entered Gmail ID and password and prints the result.
 * - "Reset": clears the Gmail ID and password fields.
 */
public class Q5_AWTForm_Events implements ActionListener {
    // Reference to the Gmail ID text field on the login form.
    private final TextField gmailField;
    // Reference to the password text field on the login form.
    private final TextField passwordField;

    // Hard-coded credentials used for validation.
    private static final String VALID_GMAIL = "admin@gmail.com";
    private static final String VALID_PASSWORD = "admin123";

    /**
     * Stores references to the login form's input fields so events can be handled.
     *
     * @param gmailField    the Gmail ID text field on the login form
     * @param passwordField the password text field on the login form
     */
    public Q5_AWTForm_Events(TextField gmailField, TextField passwordField) {
        this.gmailField = gmailField;
        this.passwordField = passwordField;
    }

    /**
     * Called automatically whenever the Login or Reset button is clicked.
     *
     * @param event the event object; its action command is the button's label
     */
    public void actionPerformed(ActionEvent event) {
        // Read the label of the button that was clicked.
        String command = event.getActionCommand();

        // Route the click to the right handler based on the button label.
        if (command.equals("Login")) {
            handleLogin(event);
        } else if (command.equals("Reset")) {
            handleReset(event);
        }
    }
}
```

```

* Event handler for the "Login" button: validates Gmail and password.
*
* @param event the Login button click event
*/
private void handleLogin(ActionEvent event) {
    // Read the Gmail ID and password typed by the user.
    String gmail = gmailField.getText().trim();
    String password = passwordField.getText();

    // Check the entered credentials against the hard-coded valid values.
    if (gmail.equals(VALID_GMAIL) && password.equals(VALID_PASSWORD)) {
        // Credentials matched - login is a success.
        System.out.println("Login successful. Welcome " + gmail + "!");
    } else {
        // Credentials did not match - login is a failure.
        System.out.println("Login failed. Invalid Gmail ID or Password.");
    }
}

/**
* Event handler for the "Reset" button: clears both input fields.
*
* @param event the Reset button click event
*/
private void handleReset(ActionEvent event) {
    // Clear the Gmail ID field so the user can type a fresh value.
    gmailField.setText("");
    // Clear the password field so the user can type a fresh value.
    passwordField.setText("");
    // Put the cursor back into the first field for the next entry.
    gmailField.requestFocus();
    // Notify the user on the console that the form was cleared.
    System.out.println("Login form reset. Fields cleared.");
}
}

```

Q5_b_Calculator_Events.java

```
// Import TextField: the non-editable display screen of the calculator.
import java.awt.TextField;
// Import(ActionEvent): the object that carries information about a button click.
import java.awt.event.ActionEvent;
// Import ActionListener: the interface that must be implemented to receive button click events.
import java.awt.event.ActionListener;

/**
 * Practical 8 - Event handling for the Q5_b_Calculator.
 *
 * Contains ONLY event handling logic - no GUI code.
 * Attaches to all 16 buttons of the calculator form, whose labels determine the action.
 *
 * Behaviour:
 * - Digits (0-9) and ".". append to the current value shown on the display.
 * - Operators (+, -, x, ÷) store the pending operator and the first operand.
 * - "=" computes the pending operation and shows the result on the display.
 */
public class Q5_b_Calculator_Events implements ActionListener {
    // Reference to the calculator's non-editable display screen.
    private final TextField display;
    // Stores the first operand of the pending calculation.
    private double firstOperand = 0;
    // Stores the pending operator ("+", "-", "x" or "÷").
    private String pendingOperator = "";
    // True when the next digit must start a fresh entry instead of appending.
    private boolean startNewEntry = true;

    /**
     * Stores the calculator display so results, digits and operators can be applied.
     *
     * @param display the calculator's display text field
     */
    public Q5_b_Calculator_Events(TextField display) {
        this.display = display;
    }

    /**
     * Called automatically when any calculator button is clicked.
     *
     * @param event the event object; its action command is the button's label
     */
    public void actionPerformed(ActionEvent event) {
        // Read the label of the button that was clicked.
        String command = event.getActionCommand();

        // Digits and the decimal point build up the number in the display.
        if (isDigitOrDecimal(command)) {
            handleDigit(command);
        }
        // The "=" button computes the pending operation.
        else if (command.equals("=")) {
            calculateResult();
        }
        // Any other label is an operator ( +, -, x, ÷ ).
        else {
            handleOperator(command);
        }
    }
}
```

```

/**
 * Handles a digit or decimal point press by appending to the display.
 *
 * @param digit the digit or "." that was pressed
 */
private void handleDigit(String digit) {
    // If this is a fresh entry, replace the old value with the new digit.
    if (startNewEntry) {
        display.setText(digit);
        startNewEntry = false;
    }
    // Otherwise prevent multiple decimal points in one number and append.
    else if (!(digit.equals(".") && display.getText().contains("."))) {
        display.setText(display.getText() + digit);
    }
}

/**
 * Handles an operator press by storing the first operand and the pending operator.
 *
 * @param operator the operator label ( +, -, x, ÷ )
 */
private void handleOperator(String operator) {
    // If a number was just entered, compute any previous pending operation first.
    if (!startNewEntry) {
        calculateResult();
    }
    // Remember the value currently on the display as the first operand.
    firstOperand = Double.parseDouble(display.getText());
    // Remember which operator the user chose.
    pendingOperator = operator;
    // The next digit starts a brand new entry.
    startNewEntry = true;
}

/**
 * Computes the pending two-operand operation and shows the result on the display.
 */
private void calculateResult() {
    // If there is no pending operator, there is nothing to calculate.
    if (pendingOperator.isEmpty()) {
        return;
    }

    // Read the second operand from the display.
    double secondOperand = Double.parseDouble(display.getText());
    // Variable that will hold the computed result.
    double result = 0;

    // Apply the correct arithmetic based on the stored pending operator.
    switch (pendingOperator) {
        case "+":
            result = firstOperand + secondOperand;
            break;
        case "-":
            result = firstOperand - secondOperand;
            break;
        case "x":
            result = firstOperand * secondOperand;
            break;
        case "÷":
            // Guard against division by zero.

```

```

        if (secondOperand == 0) {
            display.setText("Error");
            // Clear the pending operator so a mistake cannot repeat.
            pendingOperator = "";
            startNewEntry = true;
            return;
        }
        result = firstOperand / secondOperand;
        break;
    default:
        // Unknown operator: do nothing.
        return;
    }

    // Drop the trailing ".0" on whole numbers so results look clean on screen.
    if (result == Math.floor(result) && !Double.isInfinite(result)) {
        display.setText(String.valueOf((long) result));
    } else {
        display.setText(String.valueOf(result));
    }

    // Reset the calculator state after a completed calculation.
    pendingOperator = "";
    startNewEntry = true;
}

/**
 * Returns true if the given command is a digit or the decimal point.
 *
 * @param command the button label to test
 * @return true if command is 0-9 or ".", false otherwise
 */
private boolean isDigitOrDecimal(String command) {
    // Single character check: a digit, or the decimal point.
    return command.length() == 1 && (Character.isDigit(command.charAt(0)) || command.equals("."));
}
}

```

Q5_c_RegistrationForm_Events.java

```
// Import TextField: a single-line text input box on the form.
import java.awt.TextField;
// Import TextArea: the multi-line address input on the form.
import java.awt.TextArea;
// Import Checkbox: radio buttons and hobby checkboxes on the form.
import java.awt.Checkbox;
// Import Choice: the course drop-down menu on the form.
import java.awt.Choice;
// Import(ActionEvent): the object that carries information about a button click.
import java.awt.event.ActionEvent;
// Import ActionListener: the interface that must be implemented to receive button click events.
import java.awt.event.ActionListener;

/**
 * Practical 8 - Event handling for the Q5_c_RegistrationForm.
 *
 * Contains ONLY event handling logic - no GUI code.
 * Attaches to the "Save Details" and "Clear All" buttons of the registration form.
 *
 * Behaviour:
 * - "Save Details": collects every value entered on the form and prints it to the console.
 * - "Clear All": restores every field on the form to its initial/empty state.
 */
public class Q5_c_RegistrationForm_Events implements ActionListener {
    // References to the text fields on the registration form.
    private final TextField nameField;
    private final TextField fatherNameField;
    private final TextField ageField;
    // References to the gender radio buttons (male/female).
    private final Checkbox maleRadio;
    private final Checkbox femaleRadio;
    // Reference to the course drop-down menu.
    private final Choice courseChoice;
    // References to the four hobby checkboxes.
    private final Checkbox cbDrawing;
    private final Checkbox cbSinging;
    private final Checkbox cbMusic;
    private final Checkbox cbOthers;
    // Reference to the multi-line address text area.
    private final TextArea addressArea;

    /**
     * Stores every form component so the handlers can read and reset their values.
     *
     * @param nameField      the Name text field
     * @param fatherNameField the Father Name text field
     * @param ageField       the Age text field
     * @param maleRadio      the "Male" radio button
     * @param femaleRadio    the "Female" radio button
     * @param courseChoice   the Course drop-down menu
     * @param cbDrawing      the "Drawing" hobby checkbox
     * @param cbSinging      the "Singing" hobby checkbox
     * @param cbMusic        the "Music" hobby checkbox
     * @param cbOthers       the "Others" hobby checkbox
     * @param addressArea    the Address multi-line text area
     */
    public Q5_c_RegistrationForm_Events(TextField nameField, TextField fatherNameField, TextField ageField,
                                         Checkbox maleRadio, Checkbox femaleRadio, Choice courseChoice,
                                         Checkbox cbDrawing, Checkbox cbSinging, Checkbox cbMusic, Checkbox
```

```

cbOthers,
                                TextArea addressArea) {
    this.nameField = nameField;
    this.fatherNameField = fatherNameField;
    this.ageField = ageField;
    this.maleRadio = maleRadio;
    this.femaleRadio = femaleRadio;
    this.courseChoice = courseChoice;
    this.cbDrawing = cbDrawing;
    this.cbSinging = cbSinging;
    this.cbMusic = cbMusic;
    this.cbOthers = cbOthers;
    this.addressArea = addressArea;
}

/**
 * Called automatically whenever the "Save Details" or "Clear All" button is clicked.
 *
 * @param event the event object; its action command is the button's label
 */
public void actionPerformed(ActionEvent event) {
    // Read the label of the button that was clicked.
    String command = event.getActionCommand();

    // Route the click to the right handler based on the button label.
    if (command.equals("Save Details")) {
        saveDetails(event);
    } else if (command.equals("Clear All")) {
        clearAll(event);
    }
}

/**
 * Event handler for the "Save Details" button: collects and prints all form data.
 *
 * @param event the Save Details button click event
 */
private void saveDetails(ActionEvent event) {
    // Read the values typed into the name, father name and age fields.
    String name = nameField.getText();
    String fatherName = fatherNameField.getText();
    String age = ageField.getText();

    // Determine the selected gender from the radio button state.
    String gender = maleRadio.getState() ? "Male" : "Female";

    // Read the course currently chosen in the drop-down menu.
    String course = courseChoice.getSelectedItem();

    // Build a comma-separated list of the hobbies that are ticked.
    String hobbies = collectHobbies();

    // Read the multi-line address text.
    String address = addressArea.getText().replace("\n", " ");

    // Print every collected detail to the console, grouped under a header.
    System.out.println("----- SAVED REGISTRATION DETAILS -----");
    System.out.println("Name      : " + name);
    System.out.println("Father Name : " + fatherName);
    System.out.println("Age       : " + age);
    System.out.println("Gender    : " + gender);
    System.out.println("Course    : " + course);
    System.out.println("Hobbies   : " + hobbies);
}

```

```

        System.out.println("Address      : " + address);
        System.out.println("-----");
    }

    /**
     * Event handler for the "Clear All" button: resets every field on the form.
     *
     * @param event the Clear All button click event
     */
    private void clearAll(ActionEvent event) {
        // Empty the name, father name and age text fields.
        nameField.setText("");
        fatherNameField.setText("");
        ageField.setText("");

        // Restore the default radio button selection (Male).
        maleRadio.setState(true);
        femaleRadio.setState(false);

        // Reset the course drop-down back to its first entry ("Java").
        courseChoice.select(0);

        // Restore the default hobby selections (only Drawing ticked).
        cbDrawing.setState(true);
        cbSinging.setState(false);
        cbMusic.setState(false);
        cbOthers.setState(false);

        // Empty the address text area.
        addressArea.setText("");

        // Put the cursor back into the first field for the next entry.
        nameField.requestFocus();
        // Notify the user on the console that the form was cleared.
        System.out.println("Registration form cleared.");
    }

    /**
     * Builds a comma-separated String of all the ticked hobby checkboxes.
     *
     * @return the selected hobbies, or "None" if no hobby is selected
     */
    private String collectHobbies() {
        // Start with an empty list.
        StringBuilder hobbies = new StringBuilder();
        // Append each hobby label only if its checkbox is ticked.
        if (cbDrawing.getState()) {
            hobbies.append("Drawing, ");
        }
        if (cbSinging.getState()) {
            hobbies.append("Singing, ");
        }
        if (cbMusic.getState()) {
            hobbies.append("Music, ");
        }
        if (cbOthers.getState()) {
            hobbies.append("Others, ");
        }

        // If nothing was collected, report "None".
        if (hobbies.length() == 0) {
            return "None";
        }
    }

```

```
        // Strip the trailing ", " and return the list.  
        return hobbies.substring(0, hobbies.length() - 2);  
    }  
}
```